Multimedia Tools and Applications (2024) 83:44047–44066
https://doi.org/10.1007/s11042-023-17330-5

Analyzing distributed Spark MLlib regression algorithms for accuracy, execution efficiency and scalability using best subset selection approach

Piyush Sewal[1] · **Hari Singh**[1]

Received: 8 May 2023 / Revised: 16 August 2023 / Accepted: 27 September 2023 /
Published online: 17 October 2023
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2023, corrected publication 2024

Abstract
Numerous studies emphasize accuracy in machine learning regression models, yet scalability and execution efficiency are often overlooked, critical for large datasets or extensive computations. This paper introduces a scalable, distributed Spark MLlib regression model through the best subset selection approach to predict Covid-19 statistics in India, demonstrating high accuracy, scalability, and execution efficiency. Notably, limited research focuses on tree-based regression, particularly gradient boost regression, in the context of the Covid-19 dataset. The proposed work optimizes regression models for accuracy and execution time on Spark clusters of varying sizes using the best subset selection approach. Evaluation encompasses Root Mean Square Error (RMSE), Mean Absolute Error (MAE), R^2 Error for accuracy, and execution time analysis. Results indicate superior prediction accuracy in tree-based regression, with Gradient Boosted Tree Regression (GBTR) leading, and Random Forest Regression (RFR) surpassing Decision Tree Regression (DTR). Accuracy remains consistent across Python library, Spark MLlib on a single machine, and clusters of varying sizes, with Spark MLlib displaying lower execution times than Python's machine learning library on a single machine. Furthermore, execution times decrease substantially within Spark clusters, particularly for the iterative GBTR. This research uncovers scalability and execution efficiency aspects, highlighting tree-based regression's accuracy and advocating for Spark MLlib's efficacy in enhancing execution efficiency, especially across multi-node clusters.

Keywords Covid-19 · Machine Learning · Apache Spark · Cluster · Regression · Scalability · Distributed · Execution Time

✉ Piyush Sewal
piyush.sewal@gmail.com

Hari Singh
hsrawat2016@gmail.com

[1] Computer Science & Engineering Department, Jaypee University of Information Technology, Solan, Himachal Pradesh, India

1 Introduction

In the era of Information Technology, massive data is being generated from various sources on the Internet, including search engines, social media platforms, sensors, IoT devices, mobile and multimedia devices, and more. The processing of this data plays a pivotal role in decision-making [1, 2]. Big data processing frameworks, such as Hadoop, Spark, Flink, Storm, Samza, among others, have gained popularity for analyzing extensive datasets. Presently, big data analytic applications are extensively employed in social media platforms [3], enhancing healthcare systems [4], comprehending customer behaviour [5], and advancing natural language processing [6]. These applications assume a significant role in extracting knowledge from sizable datasets, which can subsequently be harnessed for early predictions, profit generation, process enhancement, and business expansion [7]. Amid the array of distributed data processing frameworks, Apache Spark stands as a prominent contender. It is particularly acclaimed for iterative data processing tasks and boasts in-memory computational capabilities that accelerate distributed data processing [8, 9].

In the realm of Bioinformatics, Apache Spark has emerged as a transformative tool offering significant advantages. With the growing volume and complexity of biological data, Spark's distributed computing capabilities enable efficient processing, analysis, scalability and interpretation of diverse datasets [10]. Its in-memory processing capabilities facilitate rapid querying and exploration of large-scale genomic and proteomic data, enhancing research efficiency. Furthermore, Spark's versatile programming interfaces and libraries enable seamless integration of various data sources and analysis methods, fostering innovative solutions in areas like variant calling, genomics data alignment, drug discovery, and biomarker identification. By using Spark's data processing power, researchers in Bioinformatics can accelerate data-driven discoveries, uncover meaningful patterns, and drive useful insights that contribute to a deeper understanding of intricate biological phenomena, a synergy that can resonate with the analysis of Covid-19 datasets to extract valuable insights for combating the pandemic [11, 12].

The global outbreak of Covid-19 brought about unprecedented challenges on a worldwide scale, demanding rapid and data-driven approaches to grasp the intricacies and effects of the virus. Governments, healthcare institutions, and researchers scrambled to devise effective mitigation measures and resource allocation strategies. Amidst this backdrop, machine learning algorithms emerged as potent tools for generating insights from the deluge of Covid-19 data [13]. Leveraging these algorithms, particularly in a distributed computing environment, offers the potential to expedite accurate predictions and scalable solutions.

While the efficacy of machine learning algorithms in predicting Covid-19 outcomes has been widely acknowledged, the complexities of the pandemic demand a nuanced approach [14]. The monumental volume and heterogeneity of data require efficient data processing framework that can operate seamlessly on large datasets. Addressing these challenges and achieving a balance between accuracy and execution efficiency has been the primary driving force behind our research endeavour. Numerous studies employ machine learning techniques for regression and classification tasks on single-machine datasets. Nevertheless, the growing dataset sizes pose a challenge due to extended processing times when using conventional Python ML libraries. Contemporary distributed machine learning libraries, such as Mahout in Hadoop and MLlib in Spark, address this concern through distributed and parallel processing capabilities. This research centres on utilizing MLlib Spark regression algorithms to predict Covid-19 statistics, a crucial endeavour in combating the

pandemic. While prior research has prioritized accurate and simple Covid-19 predictions, scalability and execution efficiency remain unexplored, especially when dealing with large datasets or extensive computations.

In this context, our study presents a comprehensive investigation into the application of Spark MLlib regression algorithms for predicting Covid-19 statistics, particularly focusing on the number of deaths. Notably, we adopt a best subset selection approach to curate a refined set of features that optimally contribute to the predictive power of the model. Through an extensive empirical analysis, we evaluate the accuracy, execution efficiency, and scalability of the selected regression algorithms. By bridging the gap between state-of-the-art distributed machine learning techniques and the critical domain of public health, our research aspires to investigate and enhance the predictive capabilities pertinent to Covid-19 outcomes.

The rest of this paper is structured as follows: Section 2 delves into the existing body of related research. Section 3 expounds upon the methodological approach adopted, encompassing both the employed regression algorithms and methodology. The dataset, along with the ensuing experimental results and their analysis, are explicated in Section 4. Discussion on the trade-off between accuracy and computational efficiency has been covered in Section 5. Finally, Section 6 wraps up the paper by summarizing our conclusions and discussing potential future research directions.

2 Related work

A multitude of researchers have harnessed non-clinical and computer science-oriented technologies, such as artificial intelligence, machine learning, deep learning, and image processing, to tackle diverse facets of the Covid-19 disease. This section initially addresses the application of the Apache Spark framework in Life Sciences and Bioinformatics. It is succeeded by an exploration of the rationale behind employing the distributed processing framework for Covid-19 forecasting. Subsequently, the section delves into previous research on Covid-19 forecasting, encompassing various methodologies such as traditional statistical and mathematical models, conventional machine learning models, and deep learning-based models. A comprehensive taxonomy of our proposed work in relation to state-of-the-art AI approaches for addressing the challenges posed by Covid-19 is presented in Fig. 1. Blue boxes indicate the areas covered in this study, while pink boxes represent domains related to the subject matter but not addressed in this study.

2.1 Spark in life sciences and bioinformatics

The study presented in [11] introduces the Spark framework and explores its applications in life sciences. Through a literature review, the versatility of Spark in managing and analyzing big data across various life science domains is demonstrated. While genomics predominates, Spark's adaptability is showcased in tackling challenges spanning metagenomics, epigenomics, biomedical imaging, signal processing, gene-disease links, and drug discovery. The survey of Spark-based applications in areas like next-generation sequencing, epigenetics, phylogeny, and drug discovery presented in [10] provides essential insights. Its findings offer a practical guide for bioinformatics researchers, enabling effective utilization of Spark in their fields. This contribution addresses genomic computational demands and promotes cutting-edge technology integration for

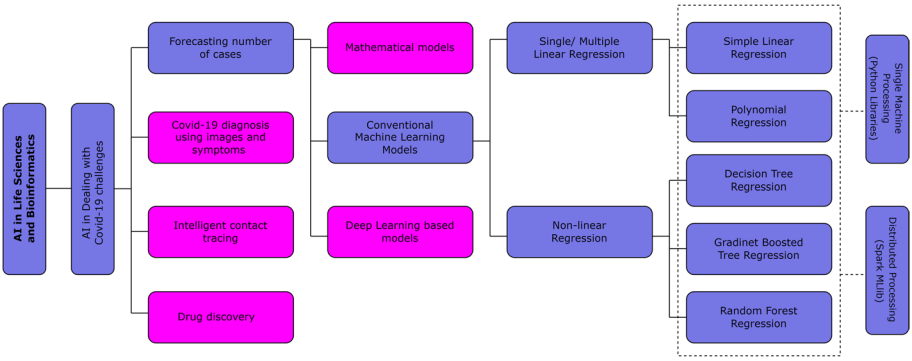

Fig. 1 A general taxonomy of the proposed work among the state-of –the-art AI approaches to deal with the challenges of Covid-19

transformative biological research. In an similar type of work, authors of [12] present ten practical guidelines to establish and utilize an Apache Spark distributed computing setup for bioinformatics data analysis which includes API selection, data compression, data partitioning, selection of appropriate cluster configuration and documentation etc. As per the claim of authors, these recommendations are tailored for novices, students, biologists, and inexperienced users, while also being relevant for experts.

2.2 Utilization of distributed processing for dealing with covid-19 challenges

Apache Spark and Hadoop have found application in diverse algorithms including Logistic Regression [15], K-Means [15]–[19], Page Rank [15, 19, 20], and KNN [21] across different datasets, demonstrating their efficacy in ensuring minimal execution time. Additionally, a machine learning model rooted in Kube Flow has been developed to compute the distance between objects through real-time video streams. These streams undergo processing via Apache Spark streaming, presenting a promising avenue for enforcing and regulating social distancing in densely populated areas. However, the model faces significant hurdles such as scalability and the cost-intensive nature of infrastructure setup [22]. To tackle the monumental data challenge posed by Covid-19, the data processing framework of Hadoop emerges as a fitting solution [23]. A classification of Covid-19 forecasting models covered in related work has been presented in Table 1.

2.3 Traditional statistical and mathematical prediction models

A mathematical predictive model scrutinizes the healthcare facilities within Italian hospitals using the Covid-19 dataset [24], while a non-parametric model is introduced utilizing a Covid-19 dataset sourced from Italy [25]. In another study, researchers examine the correlation between dependent and independent variables, employing chi-square testing to validate the relationship in the context of the spread of Covid-19 cases in South Korea [26]. Yet another crucial mathematical model employed in Covid-19 forecasting is the susceptible-exposed-infectious-recovered model (SEIR). The fusion of parameter optimization procedures and the SEIR model is showcased in [27], offering

Table 1 Classification of Covid-19 forecasting models

Model Type	Reference	Methodology
Statistical and Mathematical Model	[24]–[28]	Statistical, SEIR, Bayesian interface
Conventional Machine Learning Based Models	[34]–[37]	Linear Regression
	[33, 35, 36, 38]	Polynomial Regression
	[39]	Logistic Regression
	[30, 31, 36, 39, 41]	Decision Tree
	[34, 36, 39]–[41]	Support Vector Machine
	[31, 39]	Random Forest
	[39]	KNN
	[39, 41]	Naïve Bayes
Deep Learning Based Models	[45, 46]	Convolutional Neural Network, Recurrent Neural Network, Generative Adversarial Network, Long Short-Term Memory, Gated Recurrent Units, Graph Neural Network, Auto-Encoder-Decoder

valuable insights into China's Covid-19 situation. Furthermore, an extended iteration of the SEIR model is presented by authors who employ a Bayesian framework to estimate model parameters, contributing to classification efforts [28]. In a comprehensive effort, [29] compiles an exhaustive survey of literature, spanning various Covid-19 forecasting approaches grounded in stochastic theories and mathematical models.

2.4 Conventional machine learning based prediction models

Conventional machine learning methodologies have likewise found application in numerous preceding studies aimed at addressing the diverse challenges posed by Covid-19. Exploring alternatives to the RT-PCR testing technique, researchers have examined the viability of machine learning models for identifying Covid-19 infections based on routine blood tests [30]. In another research endeavour, the application of a machine learning-based model to accurately predict the risk levels for critically ill Covid-19 patients has been proposed [31]. Furthermore, an approach rooted in machine learning has been introduced to glean insights into public health dynamics and Covid-19 infections, utilizing data from the USA [32]. The research findings indicate a significant increase in the force of infection during the transition from mild to severe infection stages.

A polynomial regression [33] and a two-layer nested heterogeneous ensemble learning predictor [34] have been developed using Covid-19 dataset. Likewise, a machine learning-based model named Li-Muli-Poly is employed for Covid-19 statistical prediction in the USA, utilizing both linear and polynomial regression techniques. The model employs a combination of minimum mean square error, maximum likelihood ratio, and machine learning models for experimentation, leading to comprehensive analysis [35]. In [36], a supervised machine learning model integrating Linear Regression, Polynomial Regression, Decision Trees, and Support Vector Machine is proposed, demonstrating high precision in predicting Covid-19 statistics. In a separate study, linear regression is harnessed to forecast Covid-19 case growth through the variables of reproduction number, mortality rate, and recovery rate [37]. Within [38], Polynomial Regression with a degree of six is employed to foresee confirmed Covid-19 cases in weekly projections. While effective for weekly predictions, this model proves less efficient for bi-weekly and monthly forecasts.

In [39], a data mining-based prediction model is employed to anticipate recovery statistics for Covid-19 infected patients in South Korea. This is achieved by stratifying patients into distinct age groups and assessing their health risk status. The model incorporates algorithms such as Logistic Regression, Decision Tree, Support Vector, Random Forest, K-nearest neighbour, and Naive Bayes. The outcomes of these models reveal that the Decision Tree algorithm achieves the highest prediction accuracy among all the models. Presenting a practical perspective on machine learning, [40] introduces a support vector-based regression for constructing a Covid-19 prediction model encompassing twelve countries. In a separate study, Decision Tree, Naïve Bayes, and Support Vector Machines are utilized for classification purposes on a Mexican Covid-19 dataset. Notably, this application does not involve regression. It's worth mentioning that the emphasis remains on accuracy rather than execution efficiency and scalability, aspects crucial for larger datasets [41]. Furthermore, additional works focusing on Covid-19 forecasting can be found in references [42]–[44].

2.5 Deep learning based prediction models

Amidst the recent pandemic, deep learning also emerges as a promising approach to tackle Covid-19-related challenges. Numerous strategies rooted in deep learning are being employed to address a spectrum of issues associated with the pandemic. These methods encompass a diverse array of technologies such as Convolutional Neural Network (CNN), Recurrent Neural Network (RNN), Generative Adversarial Network (GAN), Long Short-Term Memory (LSTM), Gated Recurrent Units (GRUs), Graph Neural Network (GNN), and Auto-Encoder-Decoder. Authors in [45] offer a comprehensive analysis of diverse studies highlighting the significance of Deep Learning in the context of Covid-19. The study sheds light on challenges like the scarcity of research data, a concern widely faced by the research community, along with limitations such as disregard for radiological features and insufficient accessibility to interpreted medical images. Additionally, [46] presents a collection of studies exclusively cantered on the Deep Learning application for Covid-19 forecasting. This study identifies two key gaps prevalent in the existing literature: the inadequacy of research data and the mismatched forecast value. Furthermore, it underscores the necessity for more robust models integrating data with longer time horizons. Since our proposed study predominantly centres around conventional machine learning-based regression models that prioritize time efficiency and scalability along with model accuracy, we have chosen not to delve into the realm of deep learning methodologies in this context.

2.6 Analysis and refinement of Prior Covid-19 Research

In light of the aforementioned analysis of preceding Covid-19 studies, it becomes evident that a majority of traditional machine learning-based investigations have primarily employed non-iterative regression algorithms such as linear and polynomial regression. In contrast, more intricate and iterative regression techniques like decision trees and Gradient Boosting have found comparatively lesser utilization. Decision trees, for instance, exhibit superior average accuracy, are adept at handling non-linearity, and excel in addressing co-linearity issues, setting them apart from linear regression. Similarly, Gradient Boosting leverages ensemble techniques to enhance the predictive capacity of weak models, mitigating their shortcomings and yielding improved outcomes. Strikingly, the endeavour to streamline the execution times of these algorithms in response to the growing magnitude of Covid-19 datasets has not been significantly explored. While various works underline the value of distributed computing frameworks like Spark or Hadoop in achieving lower execution times, studies applying these frameworks to Covid-19 datasets remain scarce.

Thus, it becomes evident that anticipatory statistical forecasting through machine learning can play a pivotal role in mitigating risks by enabling early and thorough preparedness. Nevertheless, this endeavour is confronted by pivotal challenges such as processing capabilities, computation time, prediction accuracy, and scalability of the prediction model. While previous studies have predominantly focused on optimizing prediction accuracy, the efficient processing of extensive datasets represents a crucial step towards overall optimization.

This paper not only introduces but also validates the achievement of decreased execution times for linear and non-linear, as well as simple and iterative tree-based regression applications using the best subset methodology. Additionally, it establishes scalable

prediction models on Spark clusters of varying sizes, all cantered around the real-life Covid-19 dataset.

3 Methodology used

The methodology uses best subset selection approach in the below mentioned four steps. The LR, MLR, PR, DTR, GBTR, and RFR on Spark MLlib [47] single machine, two nodes cluster, and five nodes cluster are applied. Each algorithm computes a null model that predicts the sample mean for each observation. A separate least square regression is fit for each possible combination on p predictors for each regression algorithm. That is, p models are fit that contain exactly one predictor, all $\binom{p}{2}$ =p(p-1)/2 models that contain exactly two predictors, and so forth. Evaluation metrics and execution time are noted and analyzed. Step 4 identifies the best model on the training data for each subset of predictors on the basis of the evaluation metrics. Step 4 identifies the best model out of the obtained (p + 1) models on the basis of cross validation predictor error, adjusted R^2. The experiment uses the Covid-19 dataset of India available on Kaggle [48].

1. Apply steps 2-4 for the LR, MLR, PR, DTR, GBTR, and RFR on Python ML library and, MLlib Spark single machine, two nodes cluster, and five nodes cluster.
2. Generate a null model on no predictor that gives sample mean for each observation, M_0.
3. For k = 1 to p predictors:
   (a) Fit all p predictors based $\binom{p}{k}$ models.
   (b) Compute and record RMSE, MAE, R^2.
   (c) Compute and record the execution time.
4. Pick the best among these $\binom{p}{k}$ models on the basis of best train evaluation metrics, M_k.
5. Obtain the best model from among M_0, M_1, ..., M_p using cross validated prediction error from R^2, RMSE and MAE.

The subsequent part presents an overview of the different distributed machine learning regression algorithms used in the paper from the Spark MLlib. The workflow of the proposed prediction model has been explained in Fig. 2.

3.1 Linear regression

The linear regression makes prediction on the linear relationship between a dependent variable (y) and one or more independent variables (x). The equation of a linear regression is

$$y = a_0 + a_1 x + \varepsilon$$

where y is the dependent variable, x is the independent variable, a_0 is line intercept, a_1 is linear regression coefficient and Ɛ is the random error. Similarly, equation for multiple linear regressions is:

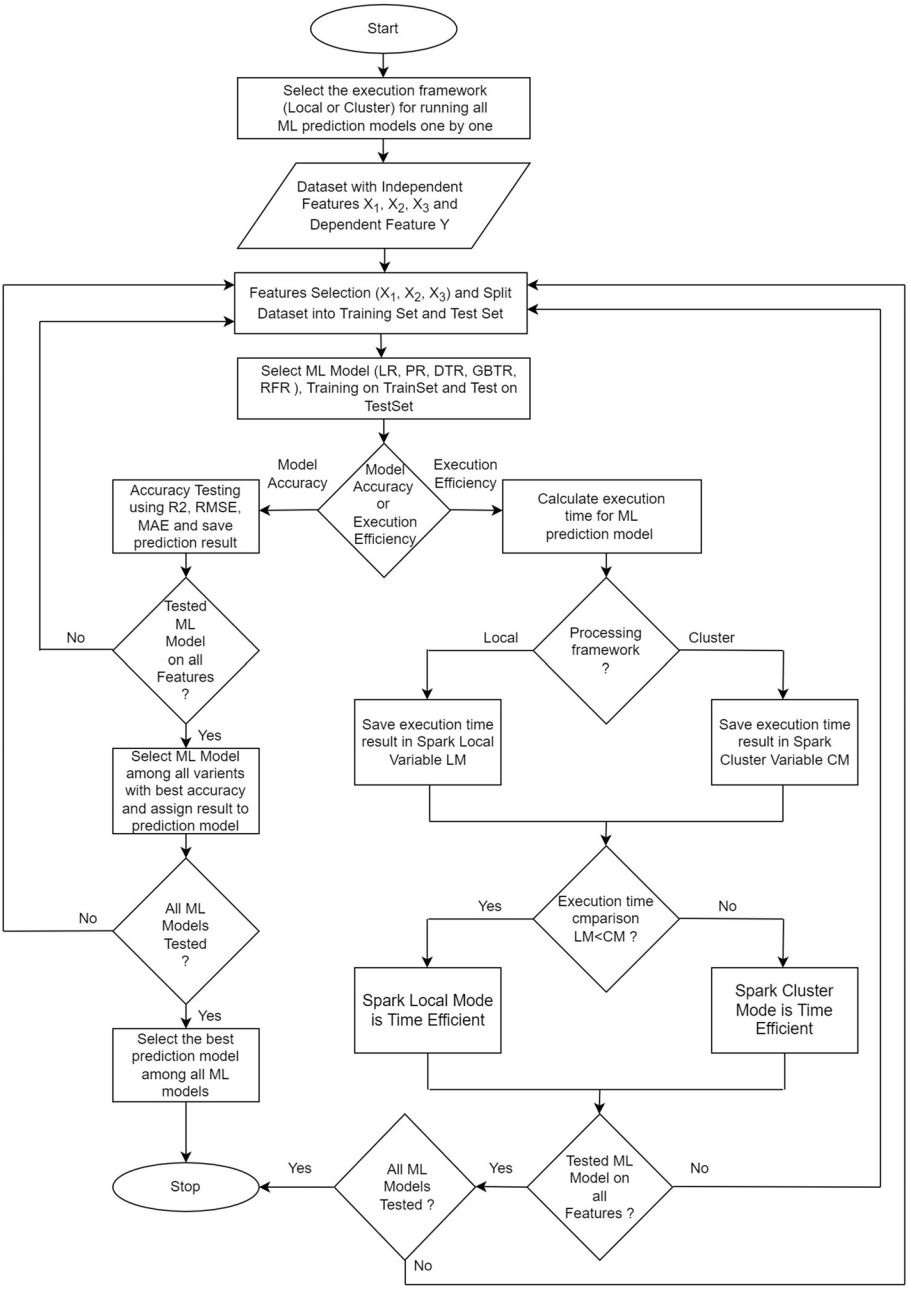

Fig. 2 Flow chart of proposed prediction model emphasizing model accuracy and execution efficiency

$$y = a_0 + a_1x_1 + a_2x_2 + a_3x_3 \ldots a_nx_n + \varepsilon$$

where, x_1,x_2……x_n are independent variables.

In our proposed linear regression model, attribute "sum(Deaths)" is taken as dependent parameter (y) and "Days", "sum(Confirmed)" and "sum(Cured) are taken as independent variables (x_1,x_2 and x_3 respectively). In simple linear regression model, only "Days" attribute is used as independent variable and in multiple regression model, "Days", "sum(Confirmed)" and "sum(Cured) are taken as independent variables. "Deaths" variable is predicted using these variables and then accuracy is tested using different accuracy metrics.

3.2 Polynomial regression

Polynomial Regression models the relationship between a dependent variable (y) and one or more independent variables (x) as the nth degree polynomial. The equation of a polynomial regression in one independent feature variable is as follows:

$$y = a_0 + a_1 x_1 + a_2 x_1^2 a_3 x_1^3 \dots a_n x_1^n$$

where y is the dependent variable, $x_1$is the independent variable, a_0 is line intercept, and a_1, a_2….a_n are the linear regression coefficients. The polynomial regression model better fits a non-linear dataset.

3.3 Decision tree regression

The decision tree is one of the most commonly used approaches for supervised learning. It divides a dataset into smaller datasets, simultaneously constructing an associated decision tree. The root node marks the initial point, which can be subsequently divided further. Decision tree features are depicted using interior nodes and leaf nodes. Cost complexity pruning is applied to larger trees to acquire the optimal trees as a function of α. Moreover, branches within the sub-tree are pruned iteratively as α's value increases. The selection of α can be accomplished through a validation set or cross-validation. The decision tree boasts ease of comprehension and implementation, necessitating minimal data cleaning. Its advantage lies in its resilience to non-linearity, as it doesn't compromise performance. The equation of Decision Tree Regression is as:

$$f(x) = \sum_{m=1}^{M} c_m . I(x \in R_m)$$

Here, f(x) denotes the predicted output for a given input x, M signifies the total number of leaf nodes in the decision tree, R_m defines the distinct region associated with each leaf node m, c_m represents the fixed value linked with the m-th leaf node and I(x∈Rm) functions as an indicator, yielding 1 if x falls within the boundaries of region R_m, and 0 if not.

3.4 Gradient boosted tree regression

The key principle behind the boosting is to improve the mistakes done by the previous learner with the help of next learner. It is similar to adaptive boosting (AdaBoost) as it also uses the decision trees ensemble for predicting the target variables but here, the depth of the tree is larger than one. In its working process, firstly, it calculates the average of target variable. So first step can be written as:

$$F_0(x) = \arg_\gamma \min \sum_{i=1}^{n} L\left(y_i, \gamma\right)$$

Here L is the loss function,y_i is the observed value, γ is the predicted value and $\arg_\gamma \min$ is the predicted value with minimum loss function. As the target column is continuous, Loss function L can be calculated as:

$$L = \frac{1}{n} \sum_{i=0}^{n} (y_i - \gamma_i)^2$$

Then, residuals, difference between the actual and predicted value, are calculated. Here predicted value is mean value calculated in the first step. After this, a decision tree is constructed and at the end, target variable is predicted using all the trees available within the ensemble. Then new residuals are calculated again and these steps are repeated as per the specification and final prediction is made at the end.

3.5 Random forest regression

The random forest is also founded on the ensemble learning technique, employing multiple decision trees. It seamlessly integrates bootstrap and aggregation in a process known as bagging. Initially, random row and feature sampling is employed to generate sample datasets for each model, a procedure referred to as bootstrapping. Random forest regression amalgamates multiple decision trees to predict the output variable's value. It begins by selecting observations and forming a training subset. Subsequently, it constructs individual decision trees for each subset. The final result is determined by the average of outputs from each subset. The equation of Random Forest can be written as:

$$yRF(x) = \frac{1}{B} \sum_{b=1}^{B} \mathrm{yb(x)}$$

Here, yRF(x) is the predicted output for a given input x using the Random Forest, B is the total number of decision trees (base models) in the Random Forest and yb(x) is the prediction from the b-th decision tree for input x.

4 Dataset, experimental results and analysis

This section discusses the description of dataset, experimental setup used, model accuracies, and model scalability and execution efficiency.

4.1 Dataset description

Before developing the prediction model, it is crucial to have an appropriate dataset that includes relevant fields and data, allowing the proposed model to effectively address similar problems. This research focuses on creating a prediction model for Covid-19-related deaths. For this purpose, the Covid-19 dataset of India from Kaggle [48] has been utilized. This dataset is available in CSV format and contains attributes such as Date, Time, State/UT, Cured, Confirmed, and Deaths. The dataset covers data for Indian States and UTs.

During the data pre-processing phase, the first step involves data cleaning, where incorrect, incomplete, corrupted, or null values in the dataset are carefully checked and corrected as needed. The subsequent step is data transformation, where certain fields are adjusted to suit the requirements of the model. To enhance data usability, the calendar dates in the dataset have been converted into numerical day values, ranging from Day 1 to Day 560. This conversion facilitates a day-by-day evaluation of Covid-19 confirmed, cured, and death cases. The final step is data reduction, where certain fields not significantly useful for the proposed prediction model are eliminated.

4.2 Experimental setup

Initially all the machines carrying 64 bit Windows OS and 4 GB RAM are installed with Apache Spark. Then, Winutils (for Hadoop) Python, Java and Anaconda Navigator are installed on all the machines. The API code is written using Python language on Jupyter Notebook. The regression algorithms are also run on a two node and five node cluster besides a single machine. The complete specification is given in Table 2.

4.3 Evaluating model accuracy

This section presents the accuracies of LR, MLR, PR, DTR, GBTR, and RFR based on varying predictors and evaluation metrics R^2, RMSE, and MAE. The results are presented in Table 3.

On the R^2 metric, all algorithms exhibit high accuracy as the number of predictor terms increases, which is expected. However, there is minimal disparity among the various algorithms with regards to R^2, prompting us to consider other metrics for clearer insights. Concerning RMSE, DT, GBTR, and RFR outperform LR and PR for one and two predictors. However, for three predictors, PR of degree three is comparable to tree algorithms, albeit

Table 2 Specification of Spark single node and cluster system

Feature	Single machine	Two node Spark cluster	Five node Spark cluster
Total number of nodes	1	2	5
Master Nodes	1	1	1
Worker Nodes	1	1	4
Executors Cores	4	4	16
Total Available Memory	1.4 GB	2.9 GB	11.6 GB
Scheduling Mode	FIFO	FIFO	FIFO
Spark Version	3.2.0	3.2.0	3.2.0
Scala Version	2.12.15	2.12.15	2.12.15
Hadoop Version	3.2	3.2	3.2
Java Version	11.0.13	11.0.13	11.0.13
IDE	Jupyter Notebook	Jupyter Notebook	Jupyter Notebook
API	Python	Python	Python
Python Version	3	3	3
OS	Windows 10	Windows 10	Windows 10
Processor	Intel Core i5-6500 CPU @ 3.20 GHz	Intel Core i5-6500 CPU @ 3.20 GHz	Intel Core i5-6500 CPU @ 3.20 GHz

Table 3 Comparison of various regression algorithms on evaluation metrics

Regression Algorithms used for predicting dependent parameter "Number of Deaths"	Independent Parameters			Degree	Accuracy Evaluation Metrics		
	Days	Confirmed	Cured		R^2	RMSE	MAE
Linear Regression	✓			NA	0.858	49046.25	39314.16
	✓	✓		NA	0.991	11689.21	8285.71
	✓	✓	✓	NA	0.995	8751.00	7478.37
Polynomial Regression	✓			2	0.943	30204.31	22920.28
	✓	✓		2	0.991	12043.57	9099.24
	✓	✓	✓	2	1.000	1609.81	**1355.47**
	✓			3	0.957	26472.73	22370.41
	✓	✓		3	1.000	8964.46	5483.27
	✓	✓	✓	3	1.000	8716.29	5431.99
Decision Tree Regression	✓			NA	0.995	7077.04	6985.49
	✓	✓		NA	0.995	7321.25	6985.49
	✓	✓	✓	NA	0.995	7630.25	6985.49
Gradient Boosted Tree Regression	✓			NA	**0.997**	**6802.91**	3584.39
	✓	✓		NA	**0.997**	**7069.24**	3911.63
	✓	✓	✓	NA	**0.996**	**7218.43**	**3887.27**
Random Forest Regression	✓			NA	0.997	6960.19	4075.11
	✓	✓		NA	0.998	7177.70	**3749.11**
	✓	✓	✓	NA	0.996	7550.30	4443.72

Values in bold indicates the best accuracy among all

with increased polynomial complexity. The accuracy doesn't see substantial improvement with further increases in the PR degree. Among the three tree-based regression models, they all display similar accuracies, with GBTR slightly outperforming RF and RF slightly outperforming DT. Similar to RMSE, all algorithms demonstrate similar behaviour on MAE, as shown in Table 3. Notably, an important observation is that algorithm accuracy remains consistent across Python library, Spark MLlib on a single machine, and clusters of varying sizes.

4.4 Model scalability and execution efficiency

Firstly, the experimental results favour Spark MLlib regression on both Spark single nodes and Spark clusters when compared to Python libraries. This is attributed to the shorter execution time of Spark MLlib regression, thanks to its in-memory computation capabilities. Notably, the accuracies of regression algorithms remain consistent across both platforms. Secondly, we perform a comparative analysis among Spark MLlib regression algorithms on a single node, a two-node cluster, and a five-node cluster based on the number of Spark jobs, the number of stages, and execution time.

Table 4 illustrates that Spark regression applications exhibit 10–15% faster execution times compared to Python regression on a single node. This difference arises because traditional Python-based execution involves frequent disk access for data

Table 4 Comparison of Spark MLlib regression algorithms on varying parameters and cluster sizes Where X1= Days, X2= Confirmed, X3= Cured

Regression Algorithm used	Independent Parameters			Number of Spark Jobs			Number of Spark Stages			Execution Time of Algorithms (s)			
	X1	X2	X3	Local Mode	Cluster Mode		Local Mode	Cluster Mode		Using Python Libraries	Spark Local Mode	Spark Cluster Mode	
				1 Node	2 Nodes	5 Nodes	1 Node	2 Nodes	5 Nodes	1 Node	1 Node	2 Nodes	5 Nodes
Linear Regression	✓			12	8	9	24	10	9	27.2	25.3	13.8	**12.5**
	✓	✓		12	7	9	24	9	9	28.3	26.2	**17.6**	18.3
	✓	✓	✓	12	9	9	24	9	9	20.2	18.6	**15.4**	21.5
Polynomial Regression	✓			14	13	13	27	13	13	16.6	**15.5**	23.2	19.6
	✓	✓		14	13	13	27	13	13	22.9	21.6	23.3	**21.2**
	✓	✓	✓	14	13	13	27	13	13	16.3	**15.1**	24.9	23.7
Decision Tree Regression	✓			12	11	11	20	17	17	24.1	21.8	8.7	**4.8**
	✓	✓		11	11	11	19	17	17	24.6	21.5	7.8	**6.6**
	✓	✓	✓	11	11	11	19	17	17	19.1	16.2	**6.8**	7.9
Gradient Boosted Tree Regression	✓			165	107	107	287	208	208	392	337	27.4	**17.8**
	✓	✓		164	107	107	286	208	208	308	272	20.2	**18.4**
	✓	✓	✓	164	107	107	286	208	208	342	307	**21.7**	23.7
Random Forest Regression	✓			21	17	17	33	23	23	127	104	16.9	**13.2**
	✓	✓		20	17	17	32	23	23	76.4	67.4	**13.2**	**13.2**
	✓	✓	✓	20	17	17	32	23	23	81	69.9	**16.2**	21.7

Values in bold indicates least execution time among different execution environment

fetching, algorithm execution, and storing intermediate results. In contrast, Spark's in-memory execution accesses the disk only once at the start, utilizing RAM until completion.

Table 4 further highlights that the number of Spark jobs and stages, as recorded from Spark WebUI, is notably lower in cluster mode execution compared to single-machine execution. This reduction ranges from 10–45% for all regression algorithms. Increased physical memory (RAM) and CPU cores in cluster mode contribute to this decrease, resulting in reduced execution times. Consequently, the execution time of LR, DTR, GBTR, and RF decreases by averages of 32%, 60%, 80%, and 90%, respectively, as also depicted in Fig. 3.

Interestingly, polynomial regression of degree two, selected for its optimal accuracy among various degrees, exhibits slight divergence. The number of jobs and stages doesn't significantly decrease, and in fact, execution time slightly increases when transitioning from a single Spark machine to a two-node cluster. However, this slight increase diminishes when shifting to a five-node cluster. This trend might be attributed to increased model complexity and network communication, potentially offsetting the benefits of distributed processing on a two-node cluster. Yet, these considerations are outweighed when additional nodes are added to the cluster.

The iterative algorithms GBTR and RFR exhibit longer execution times on Spark single machines compared to LR and PR, as also demonstrated in Fig. 4. This discrepancy arises because the task is distributed among the worker nodes of the cluster. Notably, in standalone machine execution, the number of Spark jobs and stages in iterative GBTR is ten times greater than that of the remaining four algorithms.

However, this disparity diminishes significantly, by around 40%, when these algorithms are executed on a two-node or a five-node cluster. This emphasizes the importance of a Spark cluster over single-machine processing, particularly for iterative and complex machine learning algorithms. The execution time of GBTR is reduced by over 90% when executed on a cluster. A similar reduction is evident in the RFR algorithm. However, in this case, the number of Spark jobs and stages is notably lower compared to GBTR. This difference is due to the generation of a greater number of trees and sub-trees in GBTR during different iterations.

These insights are further validated by the execution time of RFR. In light of the information presented in Table 4, Fig. 3, and Fig. 4, it can be confidently concluded that a Spark cluster offers scalability and efficient execution for time-consuming iterative regression algorithms and large datasets.

5 Balancing accuracy and computational efficiency

This section covers the discussion on the trade-off between accuracy and computational efficiency in predictive modelling, particularly in the context of bioinformatics and medical research. While our research primarily targets the scalability and execution efficiency of distributed Spark MLlib regression algorithms, it's crucial to also acknowledge scenarios where advanced models may prove advantageous despite higher computational demands.

The field of bioinformatics necessitates precise predictions, driving the need for computational resources. Understanding complex biological phenomena, disease patterns, and treatment outcomes relies on models capable of grasping intricate data relationships.

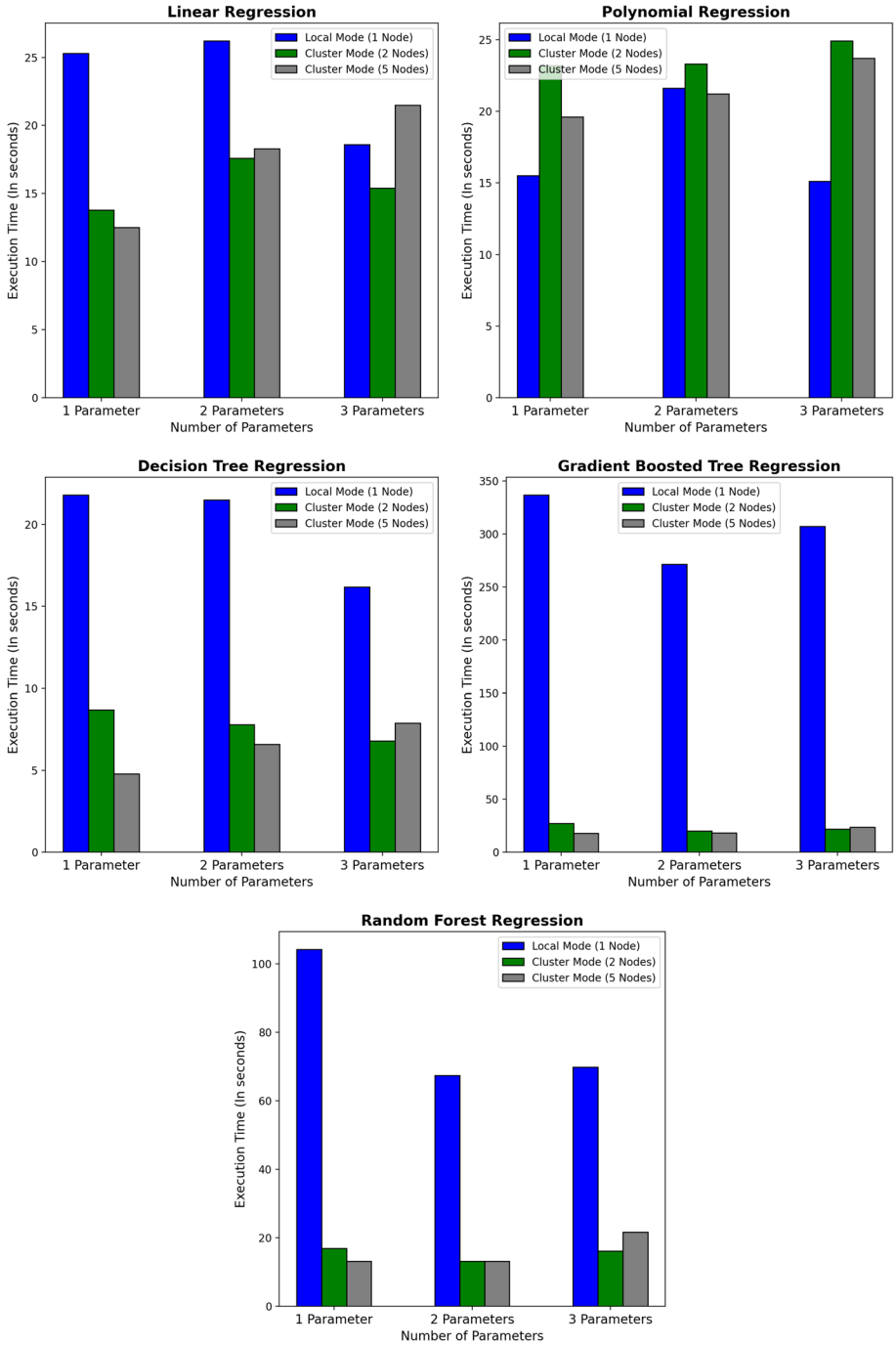

Fig. 3 Comparison of execution time of LR, PR, DTR, GBTR, and RFR for predicting Covid-19 statistics on Spark single machine, a two node cluster, and a five node cluster

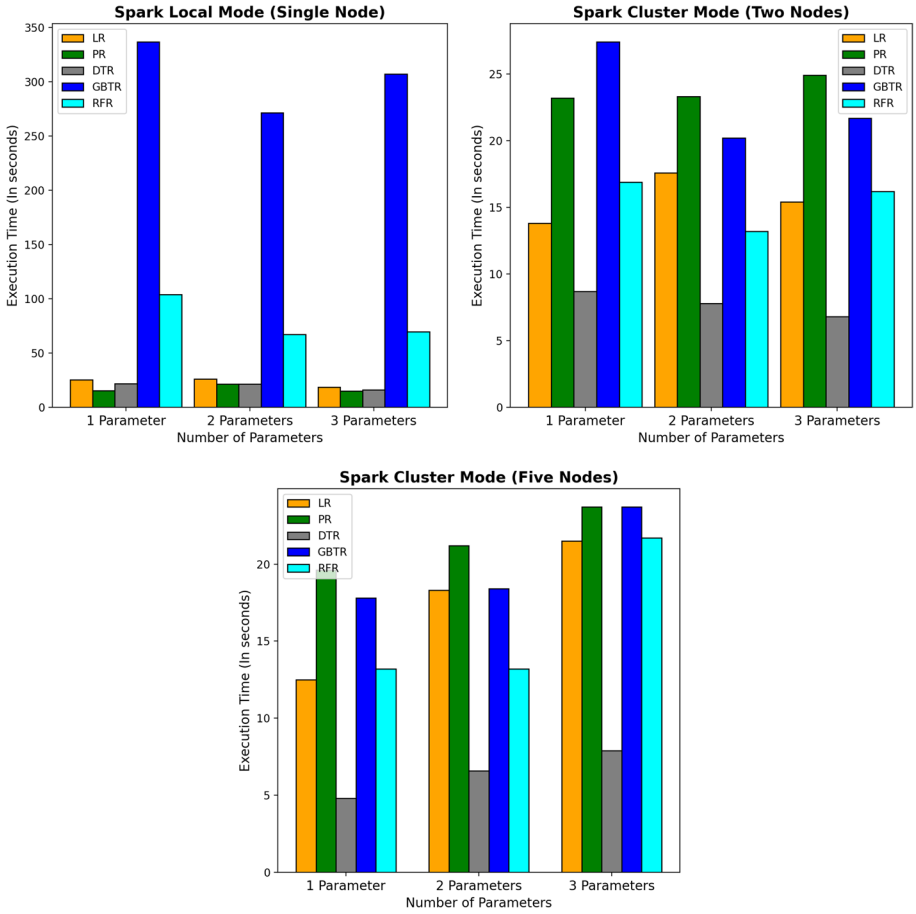

Fig. 4 Comparison of execution time of LR, PR, DTR, GBTR, and RFR for predicting Covid-19 statistics on Spark single machine, a two node cluster, and a five node cluster

Often, sophisticated models like Deep Learning approaches excel in accuracy due to their adeptness at handling complex non-linearity and high-dimensional data. However, this accuracy comes at the price of increased computational complexity and lengthier processing times. Researchers face a pivotal choice: opt for quicker results using traditional machine learning models or invest in more accurate yet computationally resource-intensive approaches.

In our Covid-19 statistics prediction research, context plays a pivotal role in shaping our modelling strategy. Urgency in pandemic situations mandates swift trend analysis, yet accuracy remains pivotal, particularly in life-critical scenarios. Our chosen approach must deftly balance achieving high prediction accuracy while efficiently handling extensive datasets. Our research, cantered around analyzing distributed Spark MLlib regression algorithms, resonates with these practical considerations. While prioritizing scalability and execution efficiency, we also acknowledge the significance of accuracy, especially in bioinformatics and medical research domains.

6 Conclusions and future scope

In the culmination of this research paper, a comprehensive analysis unfolds, scrutinizing prediction accuracies and execution times across a spectrum of Spark MLlib applications. Regression algorithms like Linear Regression, Polynomial Regression, Decision Tree Regression, Gradient Boost Tree Regression, and Random Forest Regression are evaluated using the Covid-19 dataset. This evaluation extends across diverse computing environments, encompassing a single machine, a two-node cluster, and a five-node Spark cluster. Furthermore, a comparative exploration delves into accuracy and execution efficiency disparities between traditional Python ML libraries and the Spark MLlib library. Train accuracies across various predictors are rigorously assessed using R^2, RMSE, and MAE metrics. The pinnacle models identified through a meticulous best subset selection process are subsequently subject to comparison based on cross-validation test error from R^2, RMSE and MAE.

The culmination of these investigations resoundingly substantiates Spark's unparalleled superiority in conferring elevated scalability and execution efficiency. This ascendancy is attributed to Spark's framework, distinguished by in-memory computation and distributed processing prowess. Importantly, the findings underscore the potency of Spark's distributed framework in mitigating the time-intensive nature of processing larger datasets, without compromising the accuracy of the algorithms themselves. The insights gleaned from this study hold the promise of establishing a benchmark for evaluating the prowess of advanced prediction models concerning accuracy, efficiency, and scalability metrics. In the future, our focus will pivot towards investigating the potential merits and limitations inherent in the application of more intricate methodologies, such as deep learning. This exploration seeks to amplify prediction accuracy through the synergy of distributed processing, thus augmenting both scalability and temporal efficiency.

Author contribution **Piyush Sewal:** Conceptualization, Methodology, Writing- Original draft preparation, Visualization, Investigation, Software, Validation.

Hari Singh: Supervision, Reviewing and Editing.

Both authors read and approved the final manuscript.

Funding No Funding has been received for this research work.

Data availability The datasets generated and analyzed during the current study are available from the corresponding author on reasonable request.

Declarations

Competing interests We wish to confirm that there are no known conflicts of interest associated with this publication and there has been no significant financial support for this work that could have influenced its outcome.

References

1. Singh H, Vasuja R, Sharma R (2018) A Survey of Diversified Domain of Big Data Technologies. Adv Parallel Comput 29(September):1–27. https://doi.org/10.3233/978-1-61499-814-3-1
2. Singh H, Bawa S (2017) A MapReduce-based scalable discovery and indexing of structured big data. Futur Gener Comput Syst 73:32–43. https://doi.org/10.1016/j.future.2017.03.028
3. BazzazAbkenar S, HaghiKashani M, Mahdipour E, Jameii SM (2021) Big data analytics meets social media A systematic review of techniques, open issues, and future directions. Telemat Informatics 57:101517. https://doi.org/10.1016/j.tele.2020.101517

4. Mehta N, Pandit A (2018) Concurrence of big data analytics and healthcare: A systematic review. Int J Med Inform 114(March):57–65. https://doi.org/10.1016/j.ijmedinf.2018.03.013
5. Le TM, Liaw SY (2017) Effects of pros and cons of applying big data analytics to consumers' responses in an e-commerce context. Sustain 9(5). https://doi.org/10.3390/su9050798
6. Agerri R, Artola X, Beloki Z, Rigau G, Soroa A (2015) Big data for Natural Language Processing: A streaming approach. Knowledge-Based Syst 79:36–42. https://doi.org/10.1016/j.knosys.2014.11.007
7. Janssen M et al (2015) Open and Big Data Management and Innovation. Lect Notes Comput Sci 3:200–211. https://doi.org/10.1007/978-3-319-25013-7
8. Sewal P, Singh H (2021) A Critical Analysis of Apache Hadoop and Spark for Big Data Processing, in 2021 6th International Conference on Signal Processing, Computing and Control (ISPCC). pp. 308–313. https://doi.org/10.1109/ISPCC53510.2021.9609518
9. Sewal P, Singh H (2022) A Machine Learning Approach for Predicting Execution Statistics of Spark Application. PDGC 2022 - 2022 7th Int. Conf. Parallel, Distrib. Grid Comput. pp 331–336. https://doi.org/10.1109/PDGC56933.2022.10053356
10. Guo R, Zhao Y, Zou Q, Fang X, Peng S (2018) Bioinformatics applications on Apache Spark. Gigascience 7(8):1–10. https://doi.org/10.1093/gigascience/giy098
11. Manconi A, Gnocchi M, Milanesi L, Marullo O, Armano G (2023) Framing Apache Spark in life sciences. Heliyon 9(2):e13368. https://doi.org/10.1016/j.heliyon.2023.e13368
12. Chicco D, Ferraro Petrillo U, Cattaneo G (2023) Ten quick tips for bioinformatics analyses using an Apache Spark distributed computing environment. PLoS Comput Biol 19(7):e1011272. https://doi.org/10.1371/journal.pcbi.1011272
13. Arpaci I, Al-Emran M, Al-Sharafi MA, Marques G (2021) Emerging Technologies During the Era of COVID-19 Pandemic. Studies in Systems, Decision and Control, 348. [Online]. Available: https://doi.org/10.1007/978-3-030-67716-9
14. Kamalov F, Cherukuri AK, Sulieman H, Thabtah F, Hossain A (2022) Machine learning applications for COVID-19: a state-of-the-art review, in Data Science for Genomics, Academic Press. pp. 277–289. https://doi.org/10.1016/B978-0-323-98352-5.00010-0
15. Zaharia M et al. (2012) Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing, in Proceedings of NSDI 2012: 9th USENIX Symposium on Networked Systems Design and Implementation. pp. 15–28
16. Han S, Choi W, Muwafiq R, Nah Y (2017) Impact of Memory Size on Bigdata Processing based on Hadoop and Spark, in Proceedings of the International Conference on Research in Adaptive and Convergent Systems. 2017:275–280. https://doi.org/10.1145/3129676.3129688
17. Gopalani S, Arora R (2015) Comparing Apache Spark and Map Reduce with Performance Analysis using K-Means. Int J Comput Appl 113(1):8–11. https://doi.org/10.5120/19788-0531
18. Sharma T, Shokeen DV, Mathur DS (2016) Multiple K Means++ Clustering of Satellite Image Using Hadoop MapReduce and Spark. Int J Adv Stud Comput Sci Eng 5(4):23–31 (Available: http://arxiv.org/abs/1605.01802)
19. Lin X, Wang P, Wu B (2013) Log analysis in cloud computing environment with Hadoop and Spark, Proc. 2013 5th IEEE Int. Conf. Broadband Netw. Multimed. Technol. IEEE IC-BNMT. pp. 273–276. https://doi.org/10.1109/ICBNMT.2013.6823956
20. Gu L, Li H (2013) Memory or Time: Performance Evaluation for Iterative Operation on Hadoop and Spark, in 2013 IEEE 10th International Conference on High Performance Computing and Communications & 2013 IEEE International Conference on Embedded and Ubiquitous Computing. pp. 721–727. https://doi.org/10.1109/HPCC.and.EUC.2013.106
21. Mostafaeipour A, Jahangard Rafsanjani A, Ahmadi M, ArockiaDhanraj J (2021) Investigating the performance of Hadoop and Spark platforms on machine learning algorithms. J Supercomput 77(2):1273–1300. https://doi.org/10.1007/s11227-020-03328-5
22. Melenli S, Topkaya A (2021) Real-Time Maintaining of Social Distance in Covid-19 Environment Using Image Processing and Big Data. Lect Notes Data Eng Commun Technol 76:578–589. https://doi.org/10.1007/978-3-030-79357-9_55
23. Azeroual O, Fabre R (2021) Processing big data with apache hadoop in the current challenging era of COVID-19. Big Data Cogn. Comput. 5(1):2021. https://doi.org/10.3390/bdcc5010012
24. Çakan S (2020) Dynamic analysis of a mathematical model with health care capacity for COVID-19 pandemic. Chaos, Solitons and Fractals 139. https://doi.org/10.1016/j.chaos.2020.110033
25. Singhal A, Singh P, Lall B, Joshi SD (2020) Modeling and prediction of COVID-19 pandemic using Gaussian mixture model. Chaos, Solitons Fractals 138:110023. https://doi.org/10.1016/j.chaos.2020.110023
26. AL-Rousan N, AL-Najjar H (2020) Data analysis of coronavirus COVID-19 epidemic in South Korea based on recovered and death cases. J Med Virol 92(9):1603–1608. https://doi.org/10.1002/jmv.25850

27. Sun J et al (2020) Forecasting the long-term trend of COVID-19 epidemic using a dynamic model. Sci Rep 10(1):1–10. https://doi.org/10.1038/s41598-020-78084-w
28 Prieto K (2022) Current forecast of COVID-19 in Mexico: A Bayesian and machine learning approaches. PLoS One 17(1 January):1–21. https://doi.org/10.1371/journal.pone.0259958
29. Shinde GR, Kalamkar AB, Mahalle PN, Dey N, Chaki J, Hassanien AE (2020) Forecasting Models for Coronavirus Disease (COVID-19): A Survey of the State-of-the-Art. SN Comput Sci 1(4):1–15. https://doi.org/10.1007/s42979-020-00209-9
30. Brinati D, Campagner A, Ferrari D, Locatelli M, Banfi G, Cabitza F (Aug.2020) Detection of COVID-19 Infection from Routine Blood Exams with Machine Learning: A Feasibility Study. J Med Syst 44(8):1–12. https://doi.org/10.1007/s10916-020-01597-4
31. Assaf D et al (2020) Utilization of machine-learning models to accurately predict the risk for critical COVID-19. Intern Emerg Med 15(8):1435–1443. https://doi.org/10.1007/s11739-020-02475-0
32. Magdon-Ismail M (202) Machine Learning the Phenomenology of COVID-19 From Early Infection Dynamics. pp. 1–16. https://doi.org/10.48550/arXiv.2003.07602
33 Ostertagová E (2012) Modelling using polynomial regression. Procedia Eng 48(December 2012):500–506. https://doi.org/10.1016/j.proeng.2012.09.545
34. Cui S, Wang Y, Wang D, Sai Q, Huang Z, Cheng TCE (2021) A two-layer nested heterogeneous ensemble learning predictive method for COVID-19 mortality. Appl Soft Comput 113:107946. https://doi.org/10.1016/j.asoc.2021.107946
35. Singh H, Bawa S (2021) Predicting COVID-19 statistics using machine learning regression model: Li-MuLi-Poly. Multimed Syst 28(1):1–8. https://doi.org/10.1007/s00530-021-00798-2
36. Kwekha-Rashid AS, Abduljabbar HN, Alhayani B (2021) Coronavirus disease (COVID-19) cases analysis using machine-learning applications, Appl. Nanosci., no. 0123456789. https://doi.org/10.1007/s13204-021-01868-7
37. Ghosal S, Sengupta S, Majumder M, Sinha B (2020) Diabetes & Metabolic Syndrome : Clinical Research & Reviews Linear Regression Analysis to predict the number of deaths in India due to SARS-CoV-2 at 6 weeks from day 0 (100 cases - March 14th. Diabetes Metab Syndr Clin Res Rev 14(4):311–315. https://doi.org/10.1016/j.dsx.2020.03.017
38. Yadav RS (2020) Data analysis of COVID-2019 epidemic using machine learning methods: a case study of India. Int J Inf Technol 12(4):1321–1330. https://doi.org/10.1007/s41870-020-00484-y
39. Muhammad LJ, Islam MM, Usman SS, Ayon SI (2020) Predictive Data Mining Models for Novel Coronavirus (COVID-19) Infected Patients' Recovery. SN Comput Sci 1(4):1–7. https://doi.org/10.1007/s42979-020-00216-w
40. Peng Y, Nagata MH (2020) An empirical overview of nonlinearity and overfitting in machine learning using COVID-19 data. Chaos, Solitons Fractals 139:1–15. https://doi.org/10.1016/j.chaos.2020.110055
41. Muhammad LJ, Algehyne EA, Usman SS, Ahmad A, Chakraborty C, Mohammed IA (2021) Supervised Machine Learning Models for Prediction of COVID-19 Infection using Epidemiology Dataset. SN Comput Sci 2(1):1–13. https://doi.org/10.1007/s42979-020-00394-7
42. Kumar V, Unnati S (2020) Modeling and Forecasting of COVID - 19 Growth Curve in India. Trans Indian Natl Acad Eng 5(4):697–710. https://doi.org/10.1007/s41403-020-00165-z
43. Anastassopoulou C, Russo L, Tsakris A, Siettos C (2020) Data-based analysis, modelling and forecasting of the COVID-19 outbreak. PLoS ONE 15(3):1–21. https://doi.org/10.1371/journal.pone.0230405
44. Nabi KN (2020) Forecasting COVID-19 pandemic: A data-driven analysis. Chaos, Solitons Fractals 139:1–15. https://doi.org/10.1016/j.chaos.2020.110046
45. Nayak J, Naik B, Dinesh P, Vakula K, Dash PB, Pelusi D (2022) Significance of deep learning for Covid-19: state-of-the-art review. Res Biomed Eng 38(1):243–266. https://doi.org/10.1007/s42600-021-00135-6
46. Kamalov F, Rajab K, Cherukuri AK, Elnagar A, Safaraliev M (2022) Deep learning for Covid-19 forecasting: State-of-the-art review. Neurocomputing 511:142–154. https://doi.org/10.1016/j.neucom.2022.09.005
47. Assefi M, Behravesh E, Liu G, Tafti AP (2017) Big data machine learning using apache spark MLlib, in Proceedings - 2017 IEEE International Conference on Big Data, Big Data 2017, 2018:3492–3498. https://doi.org/10.1109/BigData.2017.8258338
48. "Kaggle: Your Machine Learning and Data Science Community." https://www.kaggle.com/. Accessed 23 March 2022

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.